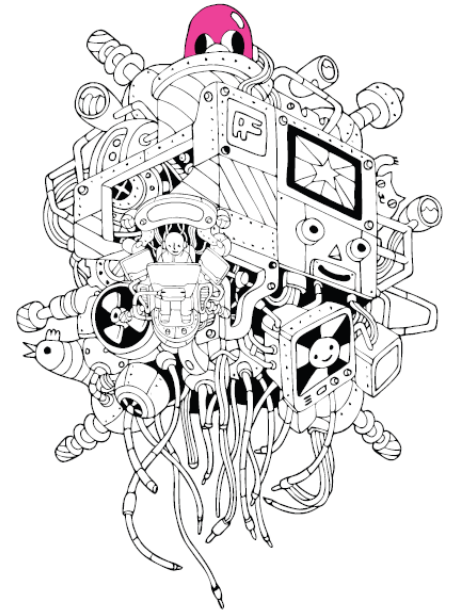


C programming



C Programming:

Chapter 08. Branching of flow according to conditions

C programming

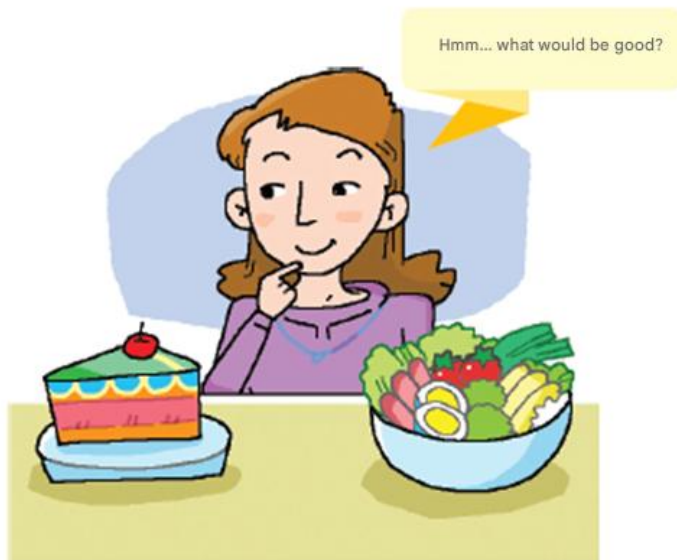


Chapter 08-1. Conditional Execution and branching of the flow

C Programming:

Conditional statement

◀ If a program does not have a selection structure, the program will always repeat the same actions.



Control statement

Control Statement

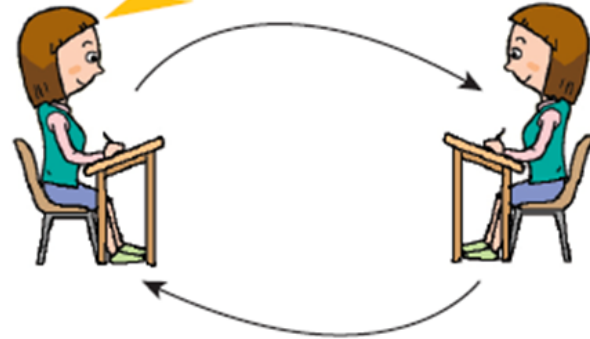
Conditional Statement

If you have good grades when you graduate, you can get a job, otherwise you have to take an exam.



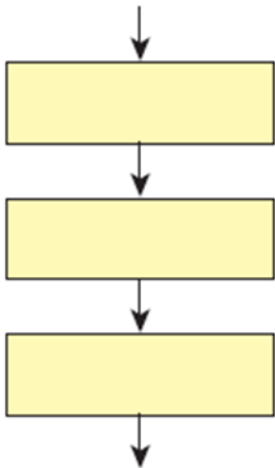
Loop Statement

You have to study repeatedly until your TOEIC score exceeds 600

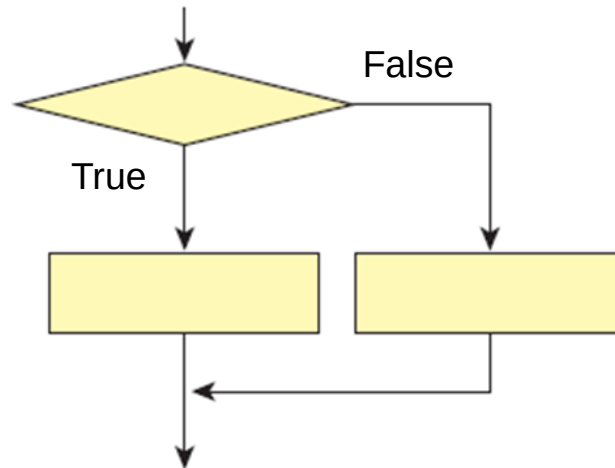


3 types of control structures

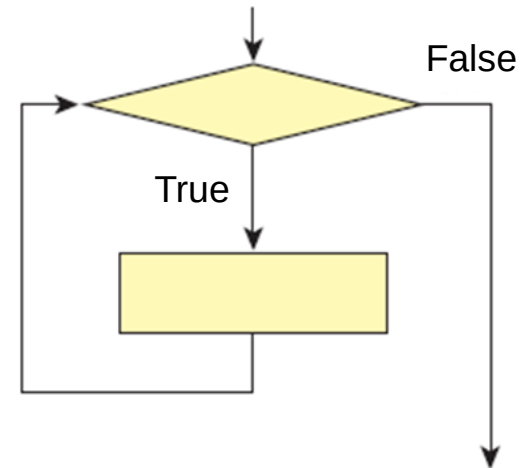
Sequential structure



Selection structure

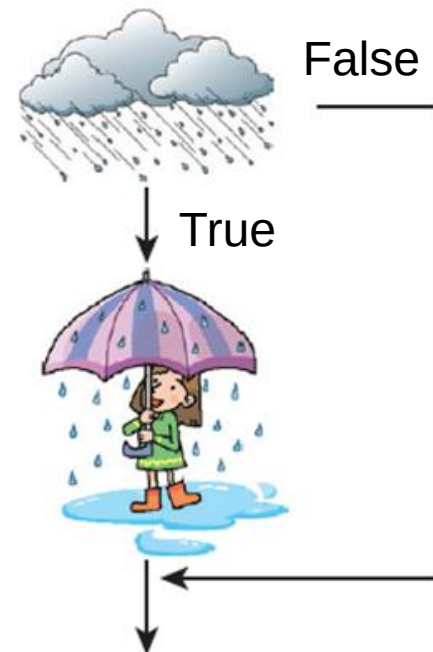
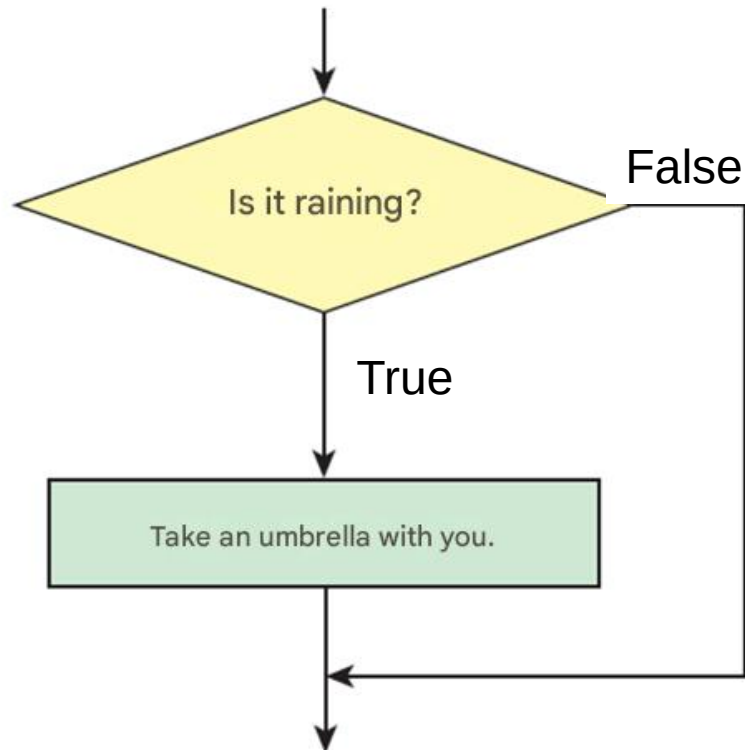


Repeating structure

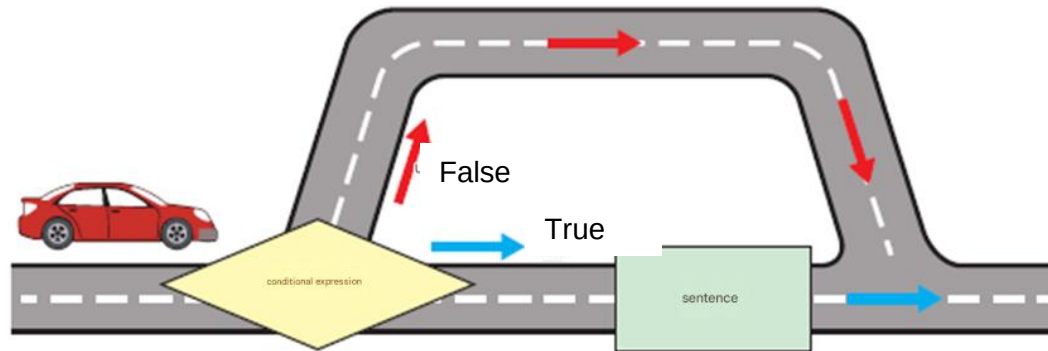
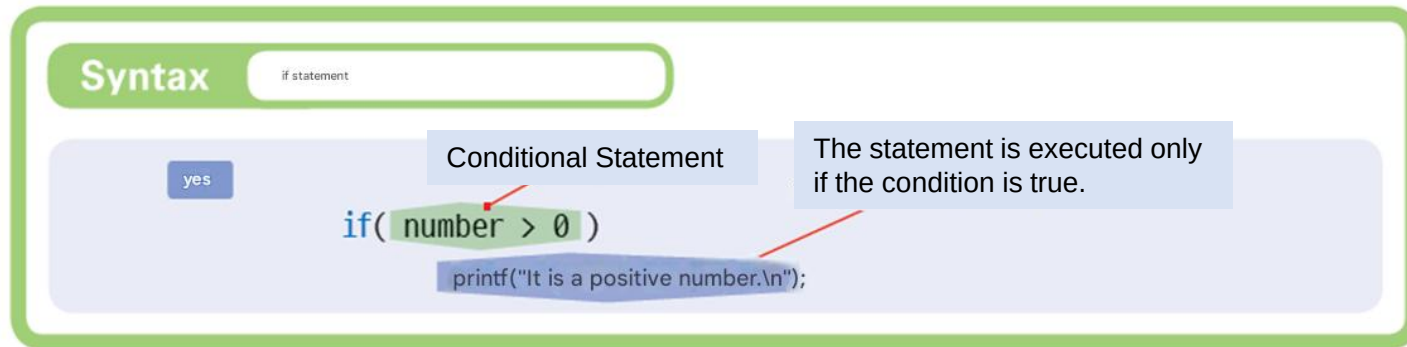


if statement

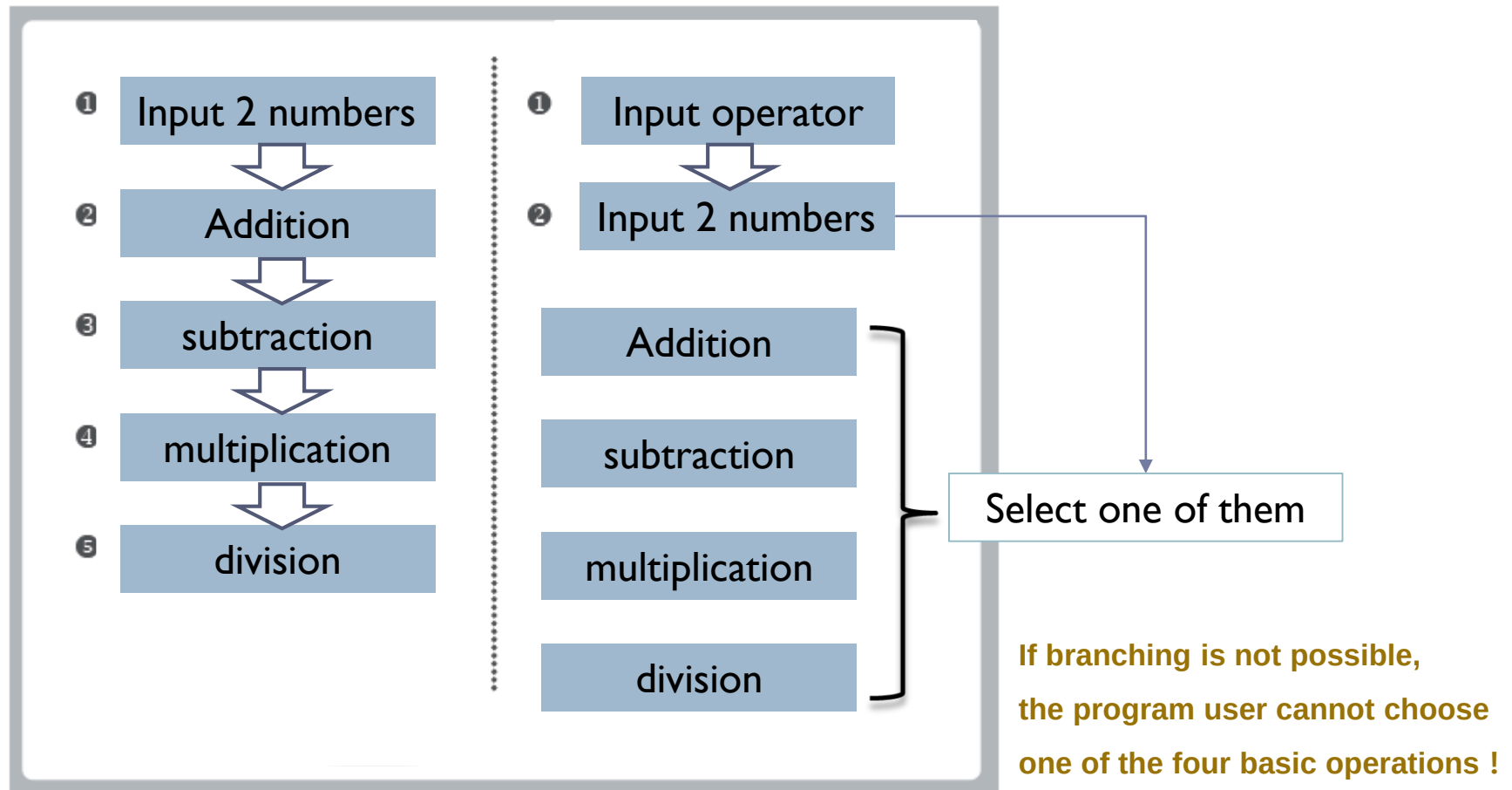
◀ In everyday life, there are many instances where decisions must be made based on conditions .



if statement structure



Why branching of the flow is necessary



When implementing a program, there are bound to be areas that need to be selectively executed depending on the situation!

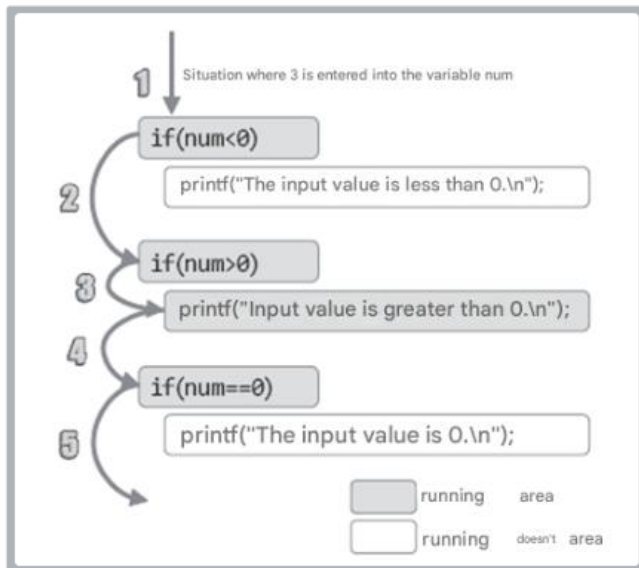
Conditional execution using if statements

Execute if num1 is greater than num2

```
if(num1>num2)
{
    printf("num1 is greater than num2.\n");
    printf("%d > %d \n", num1, num2);
}
```

If it's one line, you can omit the curly brackets.

```
if(num1>num2)
    printf("num1 is greater than num2.\n");
```



```
int main(void)
```

```
{
```

```
    int num;
```

```
    printf("Enter an integer: ");
```

```
    scanf("%d", &num);
```

```
    if(num < 0) // If num is less than 0, execute the statement below
```

```
        printf("The input value is less than 0.\n");
```

```
    if(num > 0) // If num is greater than 0, execute the statement below
```

```
        printf("The input value is greater than 0.\n");
```

```
    if(num == 0) // If num is 0, execute the statement below
```

```
        printf("The input value is 0.\n");
```

```
    return 0;
```

```
}
```

Enter an integer: 3

Execution result 1

The input value is greater than 0.

Enter an integer: 0

Execution result 2

The input value is 0.

Calculator program using if statements

ch08_if_calc.c

```
int main(void)
{
    int opt;
    double num1, num2;
    double result;

    printf("1. Addition 2. Subtraction 3. Multiplication 4. Division \n");
    printf("Select?");
    scanf("%d", &opt);
    printf("Enter two real numbers: ");
    scanf("%lf %lf", &num1, &num2);

    if(opt==1)
        result = num1 + num2;
    if(opt==2)
        result = num1 - num2;
    if(opt==3)
        result = num1 * num2;
    if(opt==4)
        result = num1 / num2;

    printf("Result: %f \n", result);
    return 0;
}
```

Now it's actually closer to a calculator program .

Due to the program's structure, only one of the four basic operations is executed. Nevertheless, even if the program user selects the addition operation, there is an unreasonable point in that a total of four conditional checks (via if statements) are performed.

if~else statement is used to resolve these irrational points.

Execution results

```
1. Addition 2.Subtraction 3.Multiply 4. Division
select? 3
Two real number inputs: 2.14 5.12
Result: 10.956800
```

A brief notation for conditional statement

standard method	concise notation
<pre>if(x != 0) printf("x is not 0.\n");</pre>	<pre>if(x) printf("x is not 0.\n");</pre>
<pre>if(x == 0) printf("x is 0.\n");</pre>	<pre>if(!x) printf("x is 0.\n");</pre>



Error

Warning: Error Caution #1

You should not put a semicolon after the conditional expression of an if statement as follows. An if statement is a single sentence made up of a conditional expression and a statement. If you write it as follows, the if statement ends with `if( x > 0 );` and the `printf` statement is executed regardless of the condition.

```
if( x > 0 );  
    printf("It is a positive number.\n");
```

Warning: Error Note #2

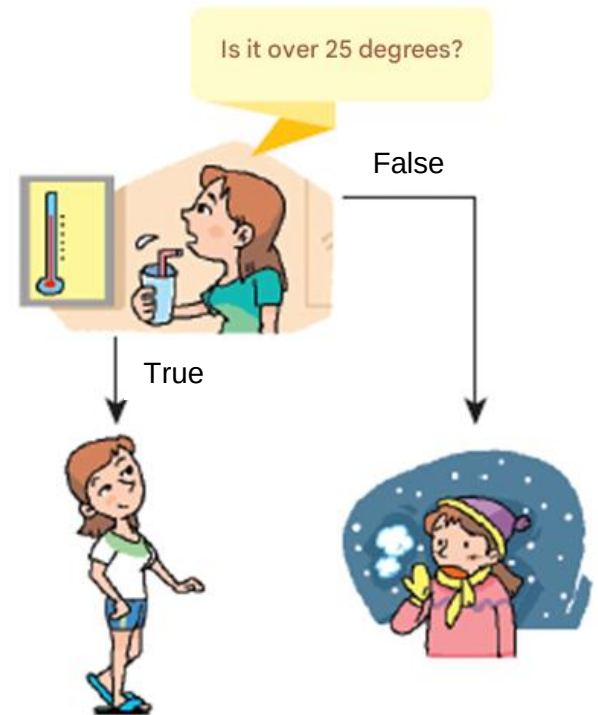
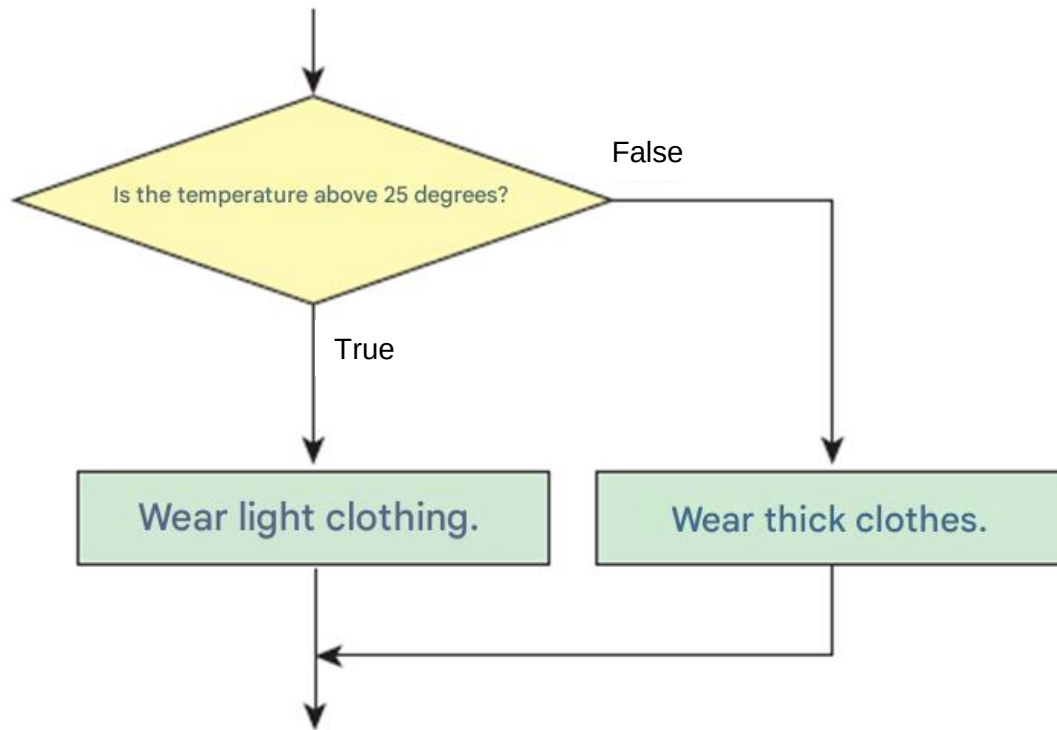
A very common mistake is to use the `=` operator instead of the `==` operator when comparing two values. In this case, no comparison is made, and the values are simply assigned to the variable. True or false is determined based on the assigned value.

```
if( x = 0 )  
    printf("x is 0.");
```

In this case, 0 is assigned to x, so it is always false. You should write `x == 0`. To avoid this error, some people write `0 == x`. If `0 = x`, a syntax error occurs.



if-else statement



if-else statement

Syntax

if-else statement

yes

Conditional expression.
`if( number > 0 )`

`printf("It is a positive number.\n");`

`else`

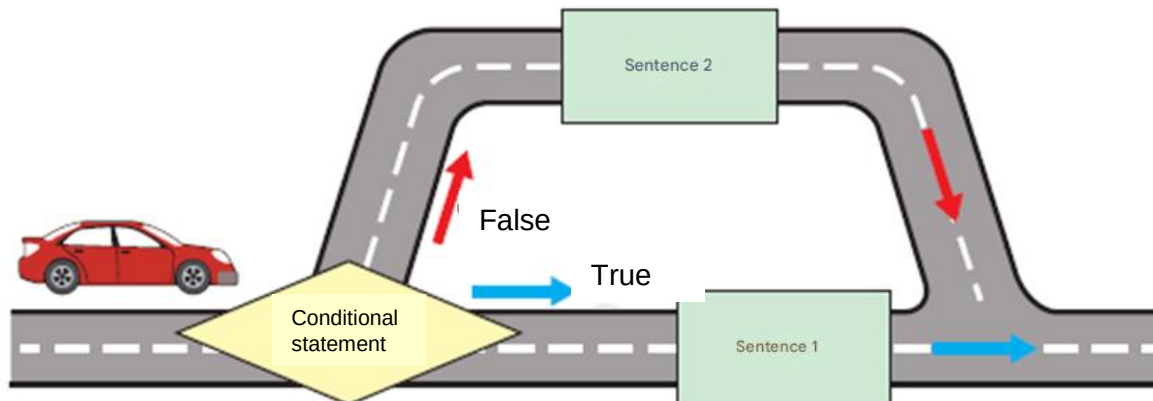
`printf("Not a positive number.\n");`

If the condition is true

Statement 1 is executed.

Otherwise, sentence 2 is true.

It is done.



Branching of flow using if-else statements

```
if(num1>num2)
```

Execute if num1 is greater than num2

```
{    // if block
    printf("num1 is greater than num2.\n");
    printf("%d > %d \n", num1, num2);
}
```

Execute if num1 is not greater than num2

```
else
{    //else block
    printf("num1 is not greater than num2.\n");
    printf("%d <= %d \n", num1, num2);
}
```

Note that the if~else statement is one statement !

Therefore, no other statements can be inserted between if and else.

```
int main(void)
{
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);
    if(num<0)
        printf("The input value is less than 0.\n");
    else
        printf("The input value is not less than 0.\n");

    return 0;
}
```

Execution results

Integer input: 7

The input value is not less than 0.

Consecutive if

Syntax

consecutive if statements

grammar

If(conditional expression 1)

 sentence 1;

else if(condition2)

 Sentence 2;

else if(condition 3)

 Sentence 3;

else

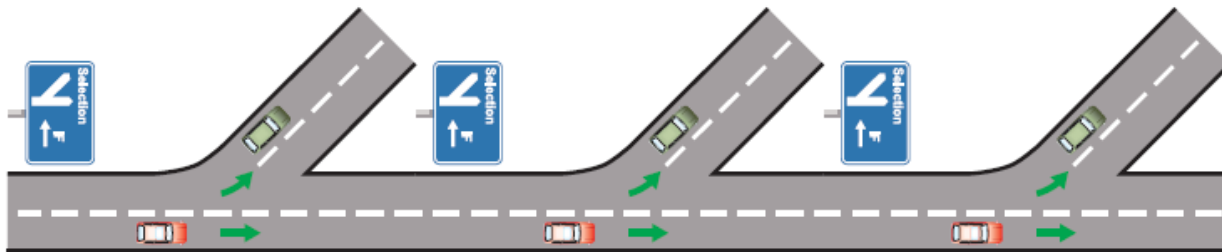
 Sentence 4;

• If condition 1 is true, statement 1 is executed.

Otherwise, if condition 2 is true, statement 2 is executed.

Otherwise, if condition 3 is true, statement 3 is executed.

Otherwise, sentence 4 is executed.



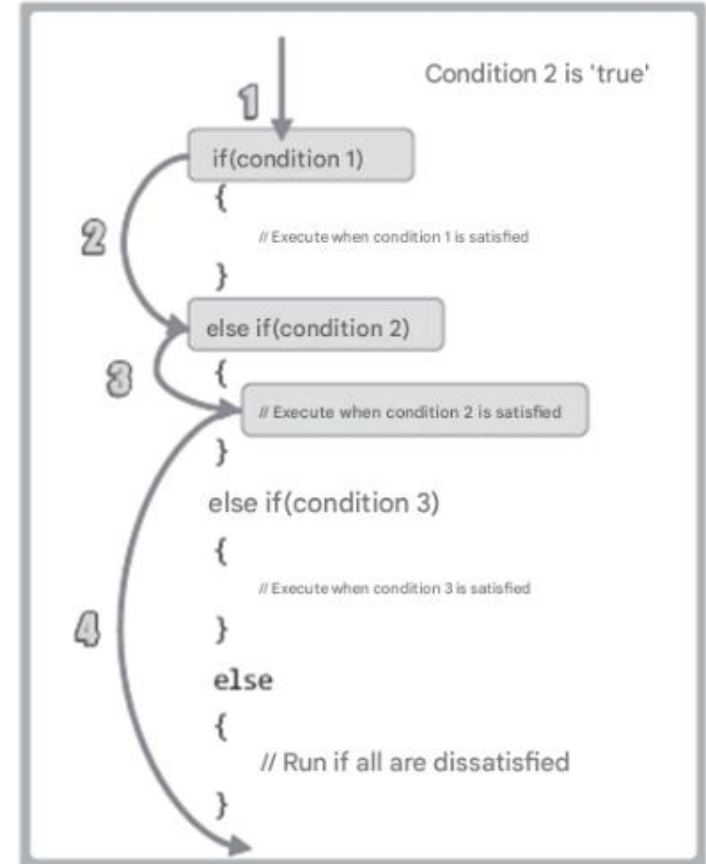
if...else structure

```
if(condition 1)
{
    // Execute when condition 1 is satisfied
}
else if(condition 2)
{
    // Execute when condition 2 is satisfied
}
else if(condition 3)
{
    // Execute when condition 3 is satisfied
}
else
{
    // Run if all are dissatisfied
}
```

if...else if...else statement structure

You can insert as many additional items as you like!

```
else if(condition 4)
{
    // Execute when condition 4 is satisfied
}
```



if...else if...else statement flow



Application of if...else

ch08_ifelse_calc.c

```
int main(void)
{
    int opt;
    double num1, num2;
    double result;
    printf("1.Addition 2.Subtraction 3.Multiplication 4.Division\n");
    printf("Select?");
    scanf("%d", &opt);
    printf("Enter two real numbers: ");
    scanf("%lf %lf", &num1, &num2);

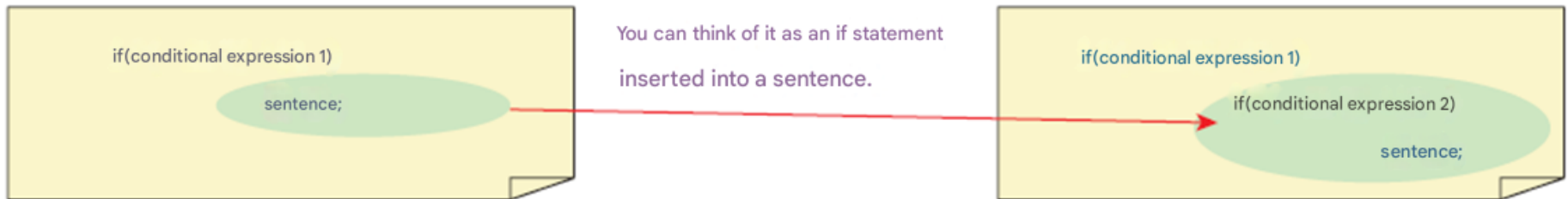
    if(opt==1)
        result = num1 + num2;
    else if(opt==2)
        result = num1 - num2;
    else if(opt==3)
        result = num1 * num2;
    else
        result = num1 / num2;

    printf("Result: %f \n", result);
    return 0;
}
```

A reasonably complete arithmetic calculator program

nested if

- if statement contains another if statement



The truth of if...else

```
if(num<0)
    printf("The input value is less than 0.\n");
else if(num>0)
    printf("Input value is greater than 0.\n");
else
    printf("The input value is 0.\n");
```

```
if(num<0)
{
    printf("The input value is less than 0.\n");
}
else
{
    if(num>0)
        printf("Input value is greater than 0.\n");
    else
        printf("The input value is 0.\n");
}
```

the if~else statement is one statement !



```
if(num<0)
    printf("The input value is less than 0.\n");
else
    if(num>0)
        printf("Input value is greater than 0.\n");
    else
        printf("The input value is 0.\n");
```

one if~else statement belongs to else.

**When there is only one sentence in a group,
the curly brackets can be omitted!**

Conditional operator: A 'ternary operator' with three operands

```
(num1>num2) ? (num1) : (num2);
```

(Condition)? data1 : data2

If the condition is true, return data1 ; if it is false, return

```
int num3 = (num1>num2) ? (num1) : (num2);
```

```
int num3 = num1; If num1>num2 is true
```

```
int num3 = num2; num1>num2 is false
```

```
int main(void)
{
    int num, abs;
    printf("Enter an integer: ");
    scanf("%d", &num);

    abs = num>0 ? num : num*(-1);
    printf("Absolute value: %d \n", abs);
    return 0;
}
```

Execution results

Integer input: -79

Absolute value: 79

Example of determining grades

- Let's write and run a program that receives students' grades and prints out their grades.

1. Receive the points from user

2. If Points ≥ 90 , then 'A'

ch08_score.c

If Points ≥ 80 , then 'B'

What about using switch?

If Points ≥ 70 , then 'C'

If Points ≥ 60 , then 'D'

If Points < 60 , then 'F'



C programming



Chapter 08-2. Omitting and escaping loops : continue & break

C Programming:

break! Let's get out of here !

```
int main(void)
{
    int sum=0, num=0;

    while(1)
    {
        sum+=num;
        if(sum>5000)
            break; // break문 실행! 따라서 반복문 탈출
        num++;
    }

    printf("sum: %d \n", sum);
    printf("num: %d \n", num);
    return 0;
}
```

The **break** statement breaks out of the loop that encloses it.

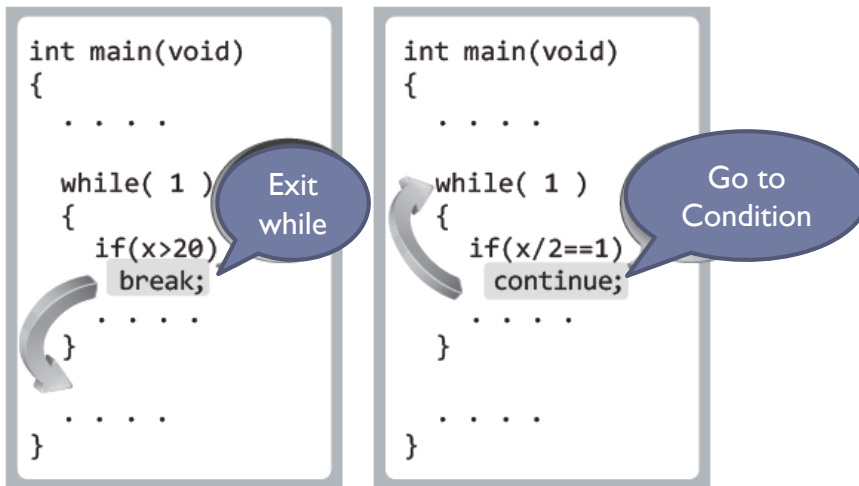
It is mainly used with the **if** statement to exit the loop when a specific condition is satisfied.

Execution results

```
sum: 5050
num: 100
```



continue! Skip the rest and check the repeat condition



The **continue** statement does not exit the loop!
It just goes up to check the repeat **conditions**.
And if the repeat condition is still 'true',
the repeat area is executed from the beginning.

```
int main(void)
{
    int num;
    printf("start! ");
    for(num=1; num<20; num++)
    {
        if(num%2==0 || num%3==0)
            continue;
        printf("%d ", num);
    }
    printf("end! \n");
    return 0;
}
```

start! 1 5 7 11 13 17 19 end!

Execution results

C programming

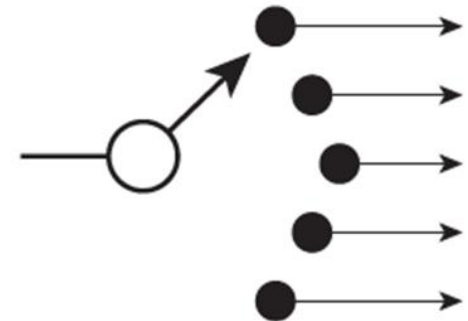
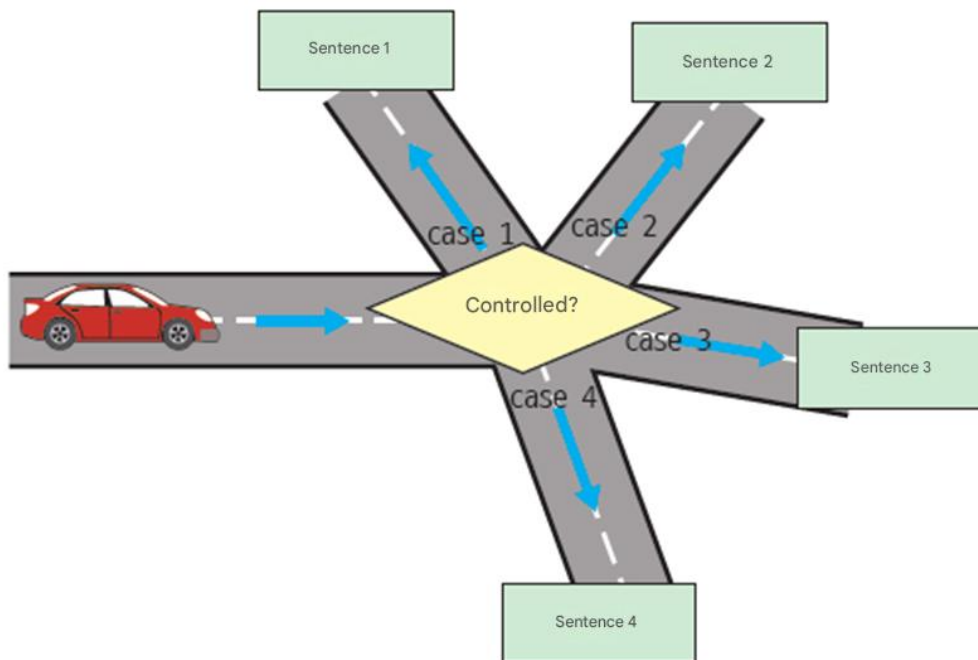


Chapter 08-3. Selective execution by switch statement and goto statement

C Programming:

switch statement

◀ A control structure that can select one of several paths depending on the value of the control expression.



switch statement

Syntax

switch statement

grammar

switch (controlled)

{

case c1:

 sentence 1;

 break;

If the value of the control expression is C1, It is executed

case c2:

 Sentence 2;

 break;

If the value of the control expression is C2, It is executed

...

default:

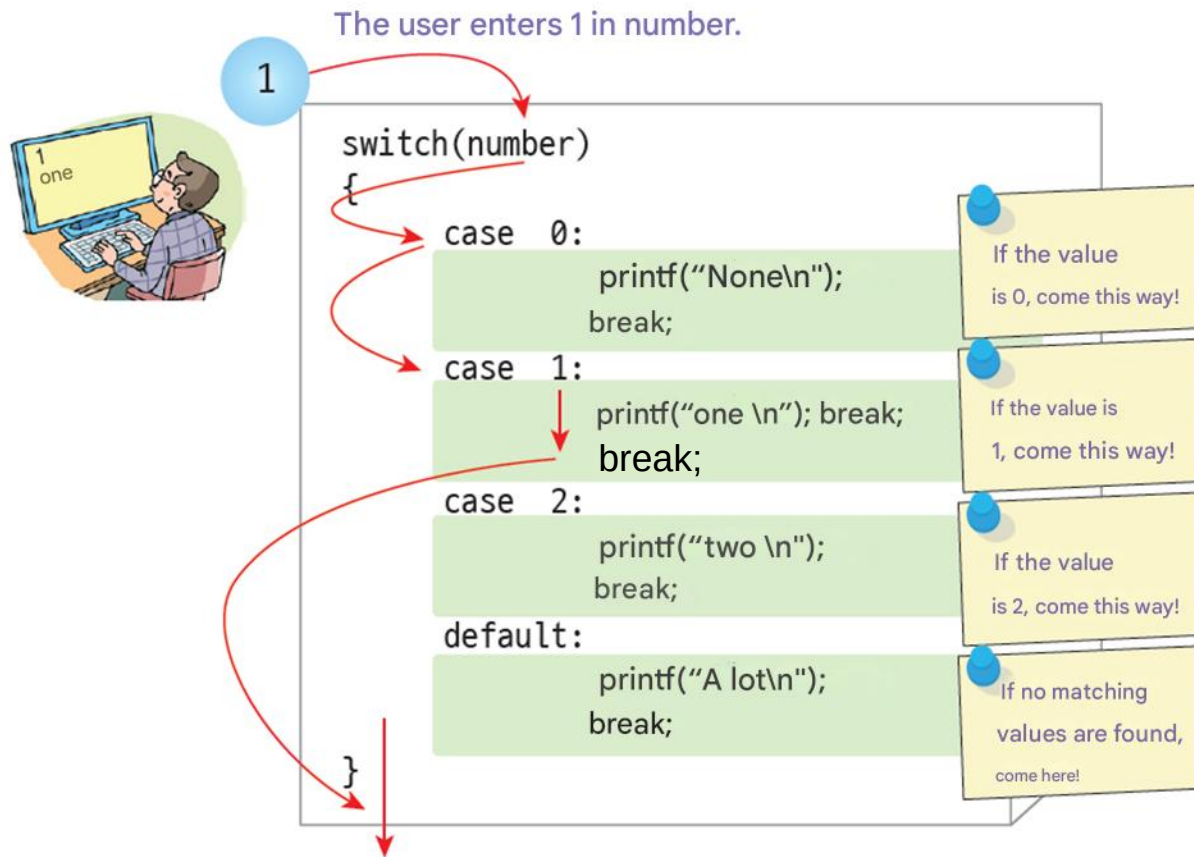
 sentence d;

 break;

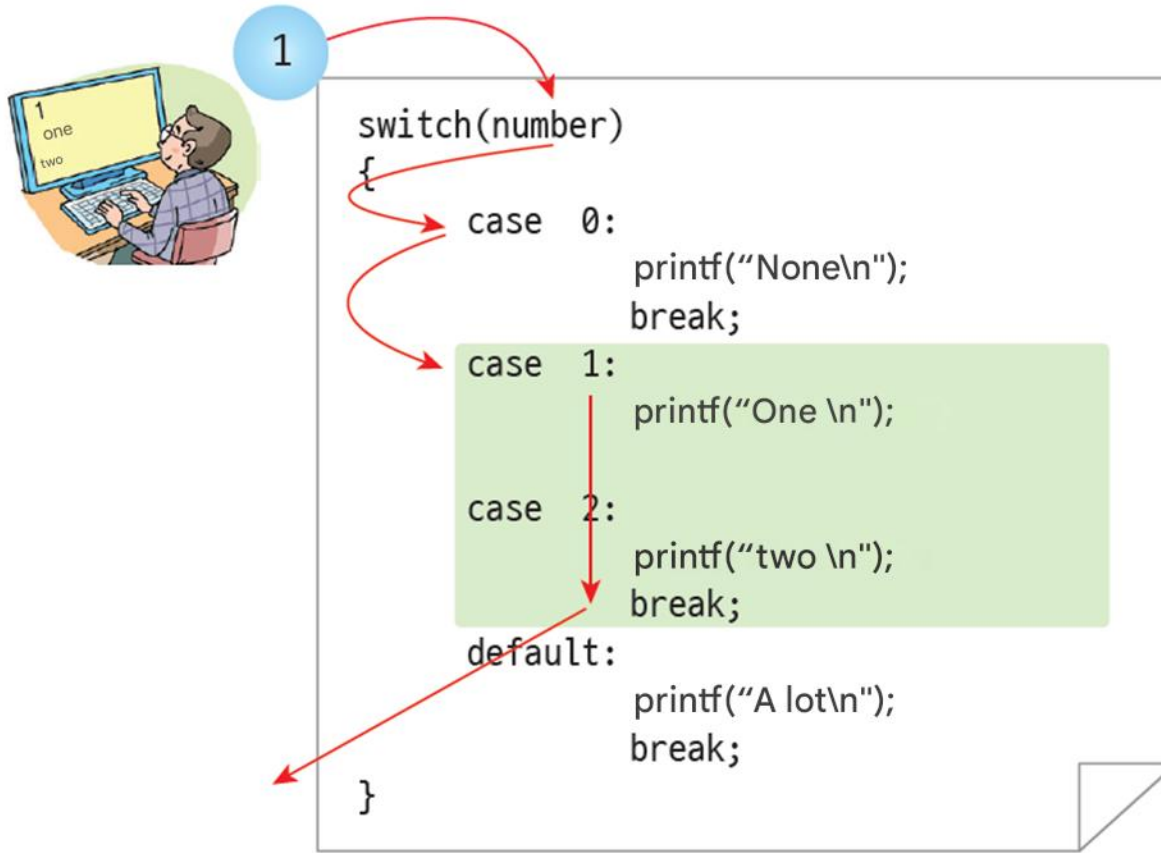
If no matching value is found, It is executed

}

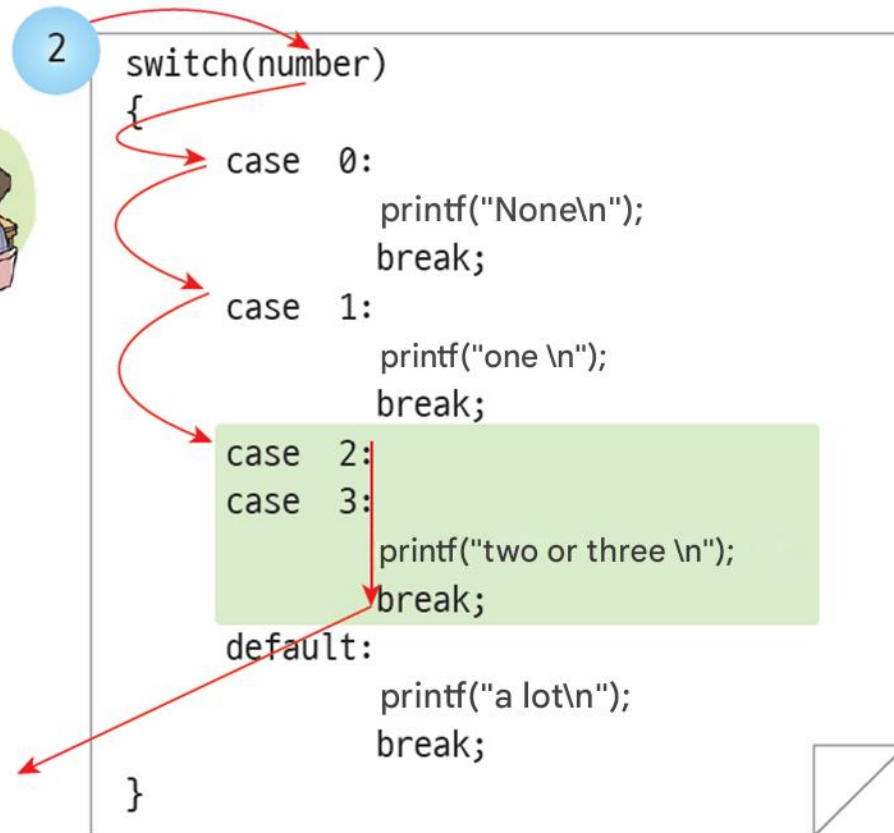
In case user enters 1



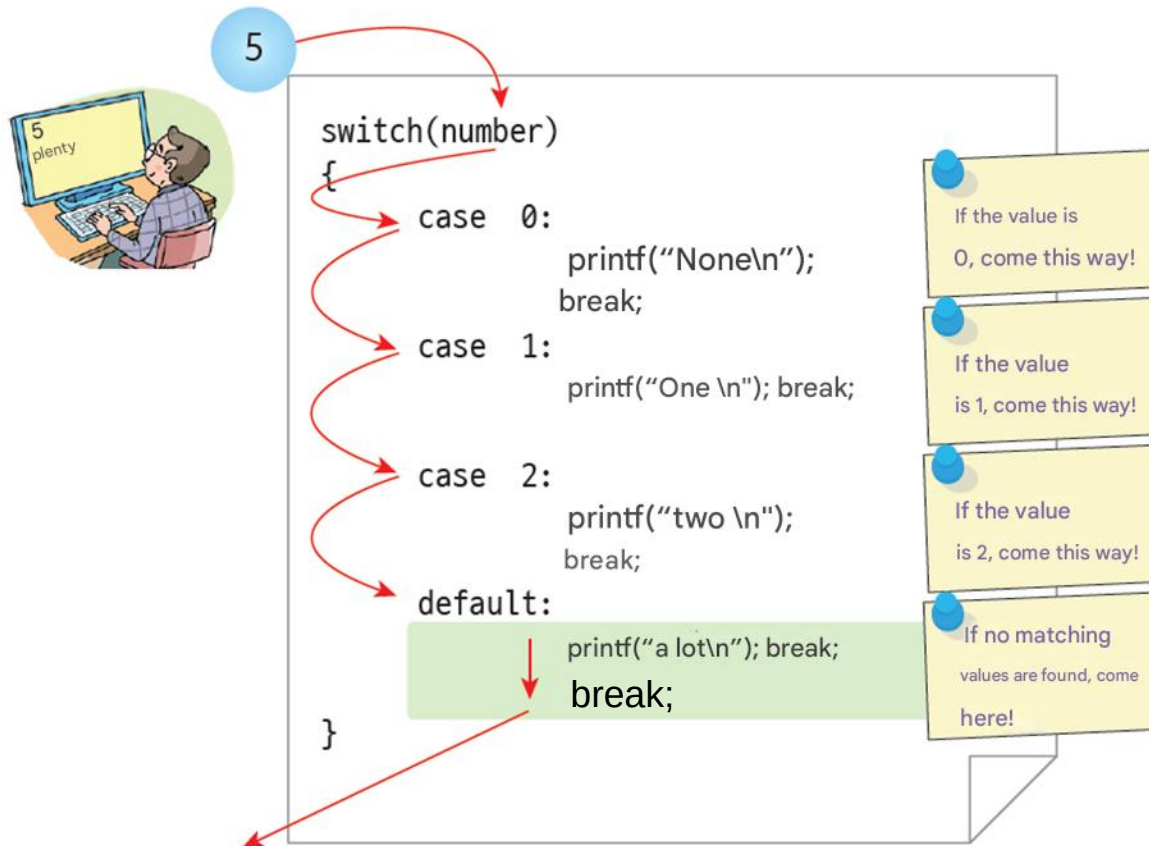
In case break is omitted



Intentional omission of break



default statement



switch statement and if-else statement

```
switch (number) {  
    case 0:  
        printf(" None \n" );  
        break;  
    case 1:  
        printf(" one \n" );  
        break;  
    case 2:  
        printf(" two \n" );  
        break;  
    default :  
        printf(" many \n" );  
        break;  
}
```



```
if ( number == 0 )  
    printf(" None \n" );  
else if ( number == 1 )  
    printf(" one \n" );  
else if ( number == 2 )  
    printf(" two \n" );  
else  
    printf(" many \n" );
```

Caution: In the switch statement

```
switch (number)
{
    case x:    // Variable cannot be used .
        printf ( " Matches x . \n " );
        break ;
    case (x+2): // Formulas containing variables cannot be used .
        printf ( " Matches formula . \n " );
        break ;
    case 0.001: // Real numbers cannot be used .
        printf ( " error \n " );
        break ;
    case 'a' : // OK! The character, int, enum is can be used .
        printf ( " character \n " );
        break ;
    case "abc": // String cannot be used .
        printf ( " string \n " );
        break ;
}
```

When indicating a range of integer

```
switch (score) {  
    case 100:  
    case 99:  
    case 98:  
    ...  
    case 90:  
        printf("It's an A.\n");  
        break;  
    ...  
}
```



```
if( score >= 90 && score <= 100 )  
    printf("It's an A.\n");
```

You can also express the range of integers, but it is cumbersome.

When indicating a range of integer

```
int iscore;
...
iscore = score/10;           // In case of integer division, the remainder is lost.
switch (iscore) {
    case 9: grade = 'A'; break; // 90-100 is an A grade
    case 8: grade = 'B'; break; // 80-89 is a B grade
    case 7: grade = 'C'; break; // 70-79 is a C grade
    case 6: grade = 'D'; break; // 60-69 is a D grade
    default: grade = 'F'; break; // 59 points or less is an F grade
}
```

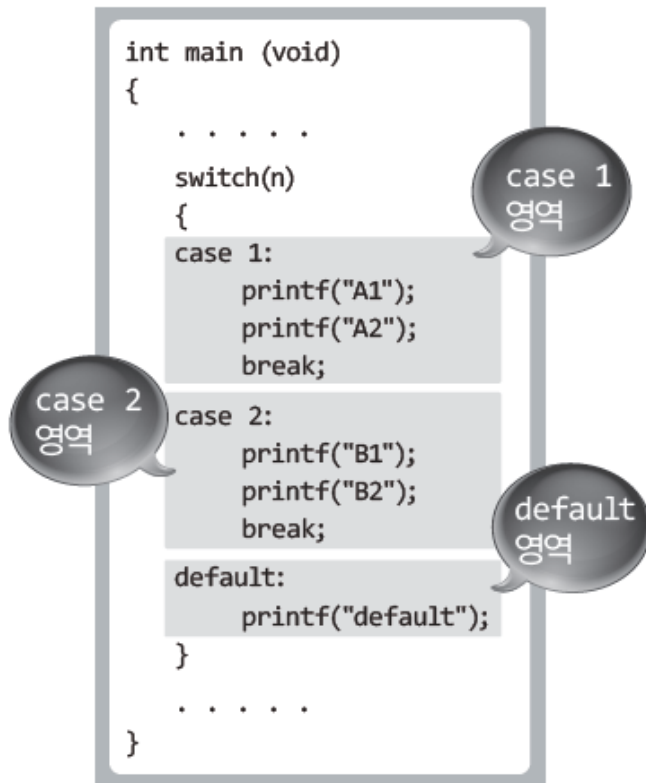


Which is more efficient: a switch statement or an if/else chain?

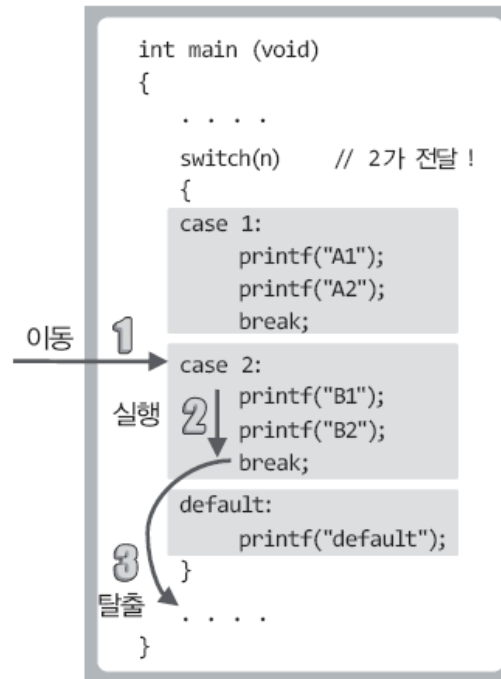
The difference is subtle. However, the switch statement is designed to be implemented efficiently with a simple jump table. Therefore, in most cases, it is better to use switch. The code is concise and probably a little more efficient.



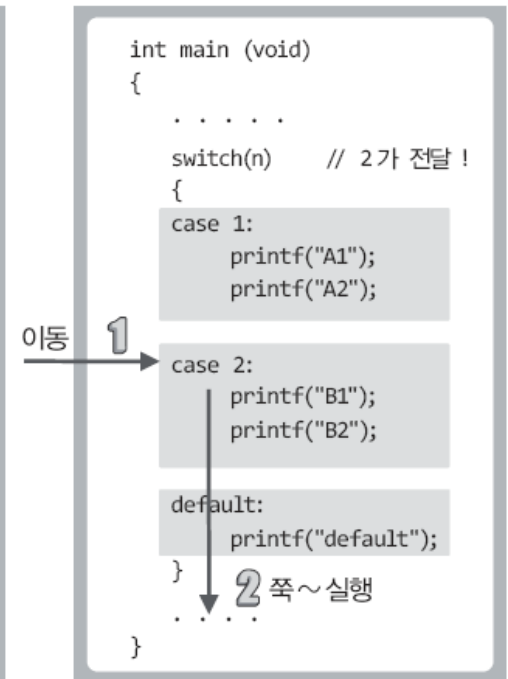
Switch statement structure and basic functions



Basic structure of switch



the inserted break statement



Examples related to switch statement

ch08_switch_calc.c

Practice !!

```
int main(void)
{
    int num;
    printf("Enter an integer between 1 and 5: ");
    scanf("%d", &num);
    switch(num)
    {
        case 1:
            printf("1 is ONE \n");
            break;
        case 2:
            printf("2 is TWO \n");
            break;
        case 3:
            printf("3 is THREE \n");
            break;
        case 4:
            printf("4 is FOUR \n");
            break;
        case 5:
            printf("5 is FIVE \n");
            break;
        default:
            printf("I don't know! \n");
    }
    return 0;
}
```

Execution result 1

```
Input an integer between 1 and 5: 3
3 is THREE
```

Execution result 2

```
Input an integer between 1 and 5: 5
5 is FIVE
```

Execution result 3

```
Input an integer between 1 and 5: 7
I don't know!
```

Configuration of a switch statement with the break statement omitted

ch08_switch.c

```
int main(void)
{
    char sel;
    printf("M AM, A PM, E Evening \n");
    printf("Input: ");
    scanf("%c", &sel);

    switch(sel)
    {
        case 'M':
        case 'm':
            printf("Morning \n");
            break;
        case 'A':
        case 'a':
            printf("Afternoon \n");
            break;
        case 'E':
        case 'e':
            printf("Evening \n");
            break; // Actually, an unnecessary break statement!
    }
    return 0;
}
```

In cases like the example on the left, the two case labels are sometimes displayed together on one line, as follows :

case 'M': case 'm':

.....

case 'A': case 'a':

.....

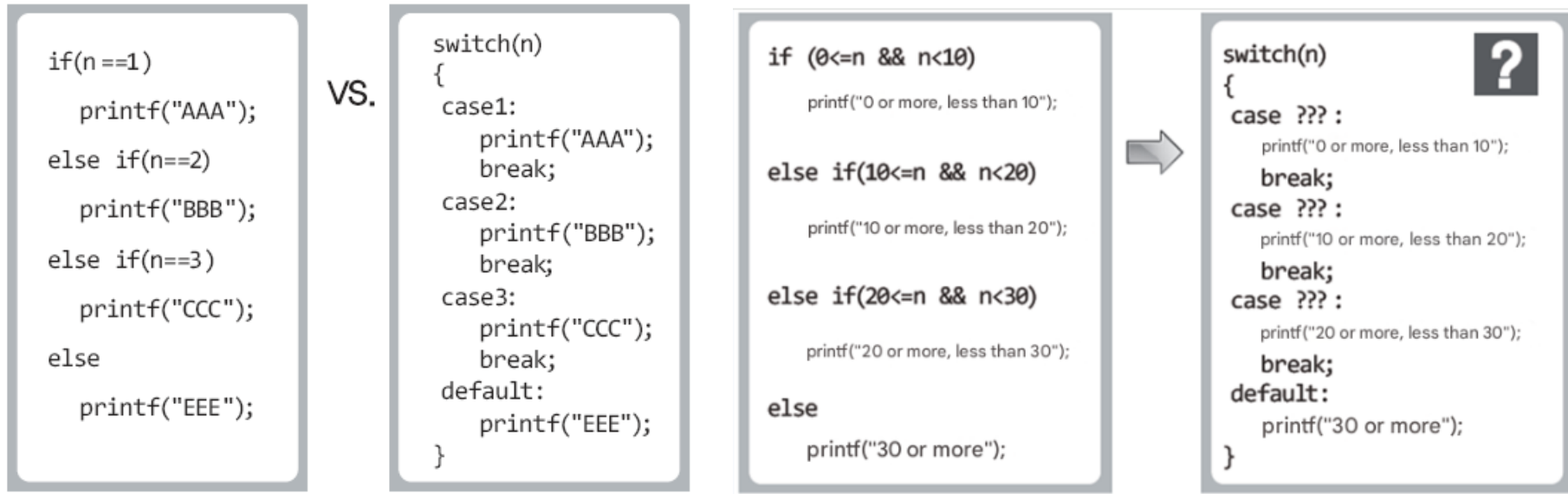
case 'E': case 'e':

.....

Execution results

M AM,A PM, E Evening
Input: M
Morning

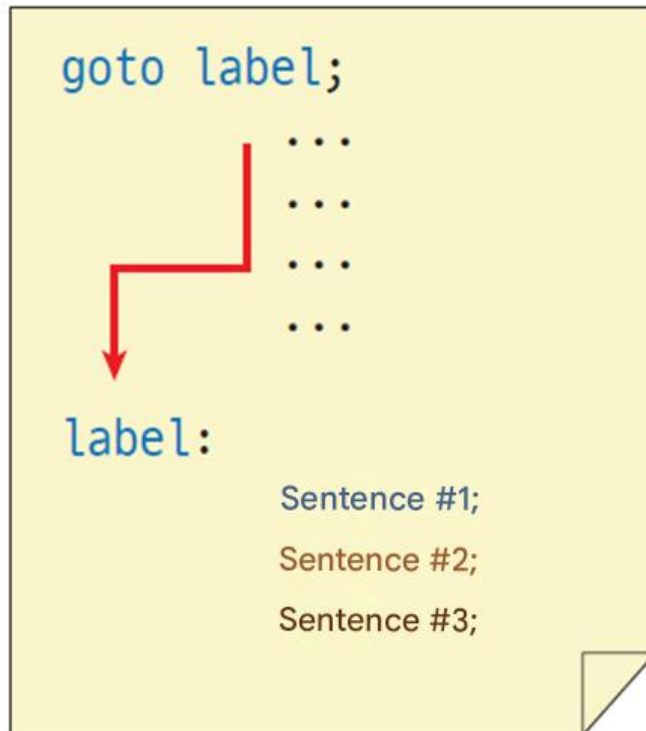
switch vs. switch if...else if...else



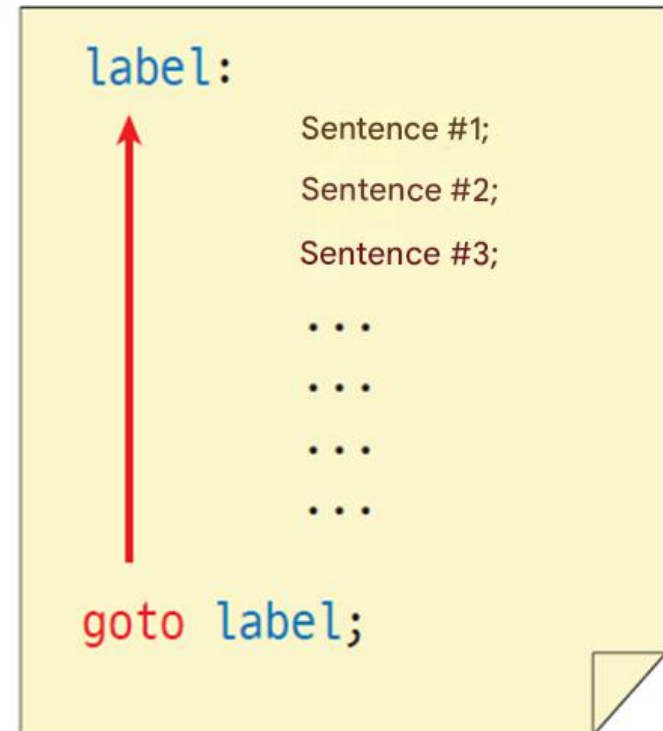
the switch statement over the if...else statement.
Because the switch statement looks more concise.

all if...else if...else statements with switch statements
It is not replaceable. **BUT !!!!**

goto door



forward looking



retrospective reference

Finally, let me introduce goto .

```
int main(void)
```

```
{
```

```
    . . . .
```

```
    rabbit: rabbit label
```

```
    . . . .
```

```
    goto rabbit: Definitely go to label rabbit !
```

```
    . . . .
```

```
}
```

**goto has many drawbacks. So, understand it,
but don't use it !**

Execution results

Enter natural number: 2

You entered 2!

```
int main(void)
```

```
{
```

```
    int num;
```

```
    printf("Enter a natural number: ");
```

```
    scanf("%d", &num);
```

```
    if(num==1)
```

```
        goto ONE;
```

```
    else if(num==2)
```

```
        goto TWO;
```

```
    else
```

```
        goto OTHER;
```

```
ONE:
```

```
    printf("You entered 1! \n");
```

```
    goto END;
```

```
TWO:
```

```
    printf("You entered 2! \n");
```

```
    goto END;
```

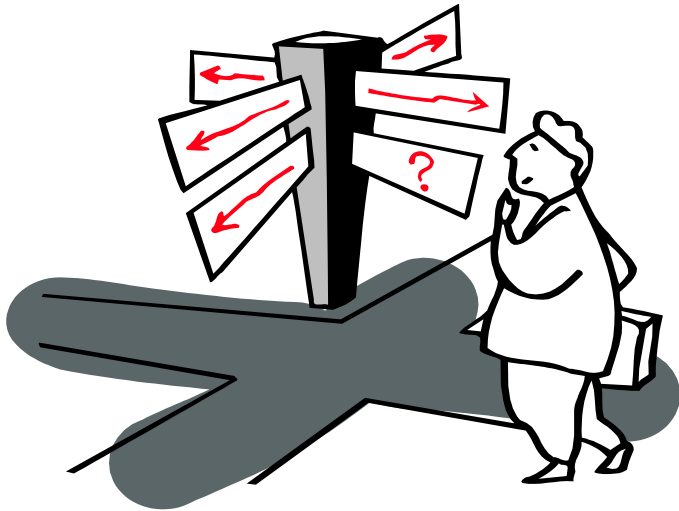
```
OTHER:
```

```
    printf("You entered 3 or another value! \n");
```

```
END:
```

```
    return 0;
```

```
}
```



Chapter 08 This That's it . Do you have any questions ?